

AI Placement Predictor — Technical Build Documentation

"Decode Employability DNA" — Institutional Career Readiness & Upskilling Engine

Hackathon Time Limit: 16 Hours | **Team Split:** Dev 1 (Frontend) / Dev 2 (Backend)

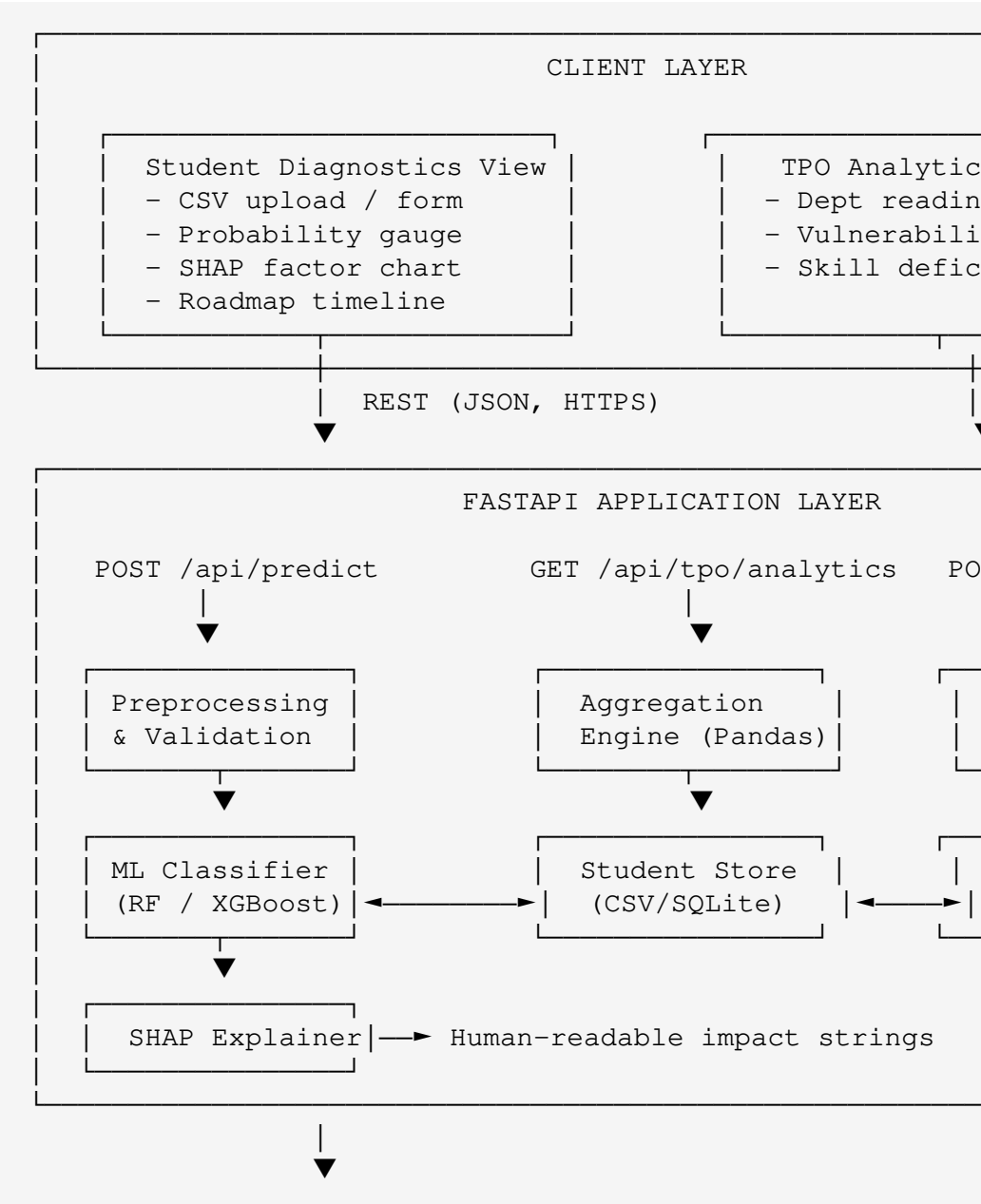
This document is the single source of truth for the build. Developer 1 and Developer 2 should each read Sections 1–2 fully, then work exclusively out of their own Part (3 or 4), syncing only through the API contract in Part C.

1. EXECUTIVE SYSTEM ARCHITECTURE & TECH STACK

1.1 System Overview

The platform has two consumer-facing surfaces (Student Diagnostics, TPO Dashboard) sitting on top of one ML-driven backend service. There is no external job-board integration — everything is scoped to the internal institutional dataset.

1.2 Data Flow Diagram (ASCII)



1.3 Frontend Tech Stack

Layer	Choice	Why (16-hr constraint)
Build tool	Vite + React 18	Instant HMR, zero-config, faster than CRA
Styling	Tailwind CSS	No context-switching to CSS files; fast iteration
Icons	Lucide-react	Tree-shakeable, consistent stroke icons
Charts	Recharts (primary), Plotly.js (only if heatmap needs it)	Recharts covers gauge/bar/line with far less boilerplate
HTTP	Axios	Interceptors for base URL + error handling
Routing	React Router v6	Two routes: /student, /tpo
State	React Query (TanStack Query) + local <code>useState</code>	Avoids hand-rolled loading/error state for every API call
CSV Parsing (client-side preview)	PapaParse	Instant client preview before upload

1.4 Backend Tech Stack

Layer	Choice	Why
-------	--------	-----

Framework	FastAPI	Auto OpenAPI docs, async, Pydantic validation
Server	Uvicorn	Standard ASGI server
Data handling	Pandas	Synthetic data gen + aggregation
ML	scikit-learn RandomForestClassifier (fallback) / XGBoost (primary)	Both handle tabular data well; XGBoost gives cleaner SHAP output
Explainability	SHAP (TreeExplainer)	Native, fast support for tree models — no LIME overhead needed
Validation	Pydantic v2	Request/response schema enforcement, matches OpenAPI contract
Persistence (MVP)	CSV + in-memory Pandas DataFrame , optional SQLite via SQLAlchemy if time permits	Avoids DB setup overhead in a 16-hr sprint

1.5 Communication Protocol

- Pure REST over HTTPS/JSON, no GraphQL (unnecessary overhead for 3 endpoints).
- CORS enabled on FastAPI for the Vite dev origin (`http://localhost:5173`).
- All responses follow the envelope:

```
{  
  "success": true,  
  "data": { ... },  
  "error": null  
}
```

- Full endpoint contracts are in **Part C**.
-

2. COMPLETE PROJECT FILE & FOLDER STRUCTURE

```
ai-placement-predictor/  
├── frontend/  
│   ├── index.html  
│   ├── vite.config.js  
│   ├── tailwind.config.js  
│   ├── postcss.config.js  
│   ├── package.json  
│   └── src/  
│       ├── main.jsx  
│       ├── App.jsx  
│       ├── router.jsx  
│       ├── pages/  
│       │   ├── StudentDiagnosticsView.jsx  
│       │   └── TpoAnalyticsDashboard.jsx  
│       ├── components/  
│       │   └── student/  
│       │       ├── ProfileInputForm.jsx  
│       │       ├── CsvUploadPanel.jsx  
│       │       └── PlacementProbabilityGauge.jsx
```

```

├── FactorContributionChart.jsx
├── UpskillingRoadmap.jsx
├── tpo/
│   ├── DepartmentReadinessCards.jsx
│   ├── CohortVulnerabilityTable.jsx
│   └── SkillDeficitHeatmap.jsx
├── common/
│   ├── Navbar.jsx
│   ├── Loader.jsx
│   ├── ErrorBanner.jsx
│   └── Card.jsx
├── services/
│   ├── apiClient.js           # Axios instance
│   ├── predictService.js      # POST /api/predict
│   ├── tpoService.js          # GET /api/tpo/anal
│   └── roadmapService.js      # POST /api/roadmap
├── hooks/
│   ├── usePredictStudent.js
│   ├── useTpoAnalytics.js
│   └── useRoadmap.js
├── context/
│   └── StudentProfileContext.jsx
├── utils/
│   ├── formatters.js
│   └── validators.js
├── assets/
│   └── theme.css
└── backend/
    ├── main.py                # FastAPI app entry
    ├── requirements.txt
    ├── config.py              # thresholds, role
    └── routers/
        ├── predict.py         # /api/predict

```

```

├── tpo.py                                # /api/tpo/analytic
├── roadmap.py                            # /api/roadmap
├── schemas/
│   ├── student_schema.py                # Pydantic request/
│   ├── prediction_schema.py
│   └── roadmap_schema.py
├── ml/
│   ├── data_gen.py                      # synthetic dataset
│   ├── train_model.py                   # offline training
│   ├── model.py                         # load + predict wr
│   ├── explainer.py                     # SHAP logic → read
│   └── artifacts/
│       ├── placement_model.pkl
│       └── feature_columns.json
├── data/
│   ├── synthetic_students.csv           # 500-row generated
│   └── role_benchmarks.json             # skill benchmark p
├── services/
│   ├── skill_gap_service.py             # gap analysis + ro
│   └── aggregation_service.py           # TPO dashboard agg
├── utils/
│   ├── validators.py
│   └── constants.py
└── README.md

```

3. PART A: FRONTEND DEVELOPER BLUEPRINT (Developer 1)

Scope note: You own everything under `/frontend`. You will consume three endpoints exactly as specified in Part C — do not wait on Dev 2's implementation; build against mocked JSON matching the contract first, then swap in the live base URL.

3.1 Component Architecture

View 1 — Student Diagnostics View (`/student`)

Component	Responsibility
CsvUploadPanel	Drag/drop or file-picker CSV upload; uses PapaParse to preview parsed rows client-side before submit; falls back to manual form if no file
ProfileInputForm	Manual entry: CGPA, 10th/12th %, backlogs, languages/frameworks, certifications (multi-select), projects count, internships count, aptitude score, soft-skill self-rating, target role dropdown
PlacementProbabilityGauge	Recharts RadialBarChart (0–100%), color banded: red <40, amber 40–60, green >60; label shows readiness status text (Ready / Near-Ready / Needs Training)

FactorContributionChart	Horizontal bar chart (Recharts BarChart with layout="vertical"), one bar per SHAP factor, colored green (positive) / red (negative), labeled with the impact string returned by the API (e.g. "+18% full-stack internships")
UpskillingRoadmap	Vertical timeline/stepper UI (phased: Phase 1 / 2 / 3), each phase shows: skill to acquire, recommended resource, estimated completion time

View 2 — TPO Analytics Dashboard (/tpo)

Component	Responsibility
DepartmentReadinessCards	Grid of cards, one per department (CSE, ISE, ECE, Mech, etc.), each showing overall readiness % and a mini progress bar
CohortVulnerabilityTable	Sortable/filterable table of students; default filter applied server-side (<60% readiness) with a client-side toggle to show all; columns: Name/ID, Department, Probability %, Status, Top Missing Skill
SkillDeficitHeatmap	Recharts (or lightweight custom SVG grid) heatmap — rows = departments, columns = skills, cell intensity = % of students lacking that skill

3.2 State Management & API Integration Hooks

Use **React Query** for all server state (caching, loading, error, retries) — do not hand-roll `useEffect` fetch logic.

```
// src/services/apiClient.js
import axios from "axios";

const apiClient = axios.create({
  baseURL: import.meta.env.VITE_API_BASE_URL || "http://localhost:3001",
  headers: { "Content-Type": "application/json" },
  timeout: 15000,
});

apiClient.interceptors.response.use(
  (res) => res.data,
  (err) => {
    const message = err.response?.data?.error || "Something went wrong";
    return Promise.reject(new Error(message));
  }
);

export default apiClient;
```

```
// src/services/predictService.js
import apiClient from "../apiClient";

export const submitStudentProfile = (payload) =>
  apiClient.post("/api/predict", payload);
```

```
// src/hooks/usePredictStudent.js
import { useMutation } from "@tanstack/react-query";
import { submitStudentProfile } from "../../services/predictService";

export function usePredictStudent() {
```

```

    return useMutation({
      mutationFn: submitStudentProfile,
    });
  }

```

```

// Usage inside StudentDiagnosticsView.jsx
const { mutate, data, isPending, isError, error } = usePr

const handleSubmit = (formValues) => {
  mutate(formValues, {
    onSuccess: (response) => {
      // response.data → { probability, status, factors[]
    },
  });
};

```

```

// src/hooks/useTpoAnalytics.js
import { useQuery } from "@tanstack/react-query";
import apiClient from "../services/apiClient";

export function useTpoAnalytics() {
  return useQuery({
    queryKey: ["tpo-analytics"],
    queryFn: () => apiClient.get("/api/tpo/analytics"),
    staleTime: 60_000,
  });
}

```

Form submission flow (Student view):

1. User fills `ProfileInputForm` OR uploads CSV via `CsvUploadPanel`.
2. On submit, payload is validated client-side (`utils/validators.js` — required fields, CGPA range 0–10, no negative counts).
3. `usePredictStudent().mutate(payload)` fires.
4. On success: populate `PlacementProbabilityGauge`, `FactorContributionChart`, and trigger POST `/api/`

`roadmap` (or consume `roadmap` embedded in the same response — see Part C, both patterns are supported).

5. On error: render `ErrorBanner` with the interceptor's normalized message; never let the raw axios error reach the UI.

TPO Dashboard flow:

1. `useTpoAnalytics()` fires on mount.
2. `CohortVulnerabilityTable` reads `data.vulnerable_students` by default; a local toggle switches to `data.all_students` (kept in component state, not refetched).
3. `SkillDeficitHeatmap` reads `data.skill_deficit_matrix` directly — no client-side recomputation.

3.3 UI/UX Styling & Theme Guidelines

Palette (Tailwind config extension):

```
// tailwind.config.js (excerpt)
theme: {
  extend: {
    colors: {
      brand: {
        50: "#eef2ff", 100: "#e0e7ff", 500: "#4f46e5", 600: "#413381",
      },
      status: {
        ready: "#16a34a", // green
        near: "#d97706", // amber
        risk: "#dc2626", // red
      },
    },
  },
},
}
```

- **Layout hierarchy:** Top-level `Navbar` (fixed, brand + view

switcher) → main content in a `max-w-7xl mx-auto px-6 py-8` container → cards use `rounded-2xl shadow-sm border border-slate-200 p-6`.

- **Typography:** System font stack (`font-sans` default Tailwind) for speed; headings `font-semibold text-slate-900`, body `text-slate-600`.
 - **Responsive grid:** Student view = single column on mobile, `grid-cols-1 lg:grid-cols-2` for gauge+chart pairing. TPO dashboard = `grid-cols-1 md:grid-cols-2 xl:grid-cols-4` for department cards, full-width table/heatmap below.
 - **Status color convention (use everywhere, consistently):** green = Ready ($\geq 75\%$), amber = Near-Ready (60–74%), red = Needs Training ($< 60\%$). This threshold logic should live in one shared `utils/formatters.js` function (`getReadinessStatus(probability)`) — do not duplicate it across components.
 - **Loading/empty/error states:** Every data-driven component must handle all three explicitly (`Loader`, an empty-state message, and `ErrorBanner`) — non-negotiable for demo stability.
-

4. PART B: BACKEND DEVELOPER BLUEPRINT (Developer 2)

Scope note: You own everything under `/backend`. Build and validate the ML pipeline offline first (`train_model.py`), then wire it into FastAPI routes. Do not block on the frontend — expose `/docs` (FastAPI auto Swagger) so Dev 1 can self-serve the contract.

4.1 Database Schema & Data Models

Given the 16-hour constraint, persistence is a flat CSV loaded into a Pandas DataFrame at startup (SQLite optional upgrade if time remains — schema below works for either).

Student (core profile + academic):

Field	Type	Notes
student_id	string (PK)	e.g. STU_0001
name	string	synthetic/anonymized
department	enum	CSE, ISE, ECE, MECH, CIVIL
semester	int	1–8
cgpa	float	0.0–10.0
tenth_pct	float	0–100
twelfth_pct	float	0–100
backlogs	int	0–10

SkillProfile (technical + experience):

Field	Type	Notes
student_id	string (FK)	
known_languages	list[string]	e.g. ["Python", "Java"]
certifications	list[string]	
projects_count	int	
internships_count	int	
open_source_contributions	int	
aptitude_score	float	0–100
soft_skill_rating	float	0–10 (self/assessed)
hackathons_attended	int	

leadership_roles	int	
------------------	-----	--

AssessmentBenchmark (per target role, static config — `role_benchmarks.json`):

Field	Type	Notes
role_name	string	e.g. Full-Stack Developer
required_skills	list[string]	
min_cgpa	float	
min_projects	int	
min_apptitude_score	float	
weight_profile	dict	feature→weight used to seed synthetic labels

ReadinessOutput (model output, not persisted unless caching predictions):

Field	Type	Notes
student_id	string	
placement_probability	float	0–100
readiness_status	enum	Ready / Near-Ready / Needs Training
target_role_scores	dict	role → confidence %
shap_factors	list[object]	{feature, impact_pct, direction, readable_string}
missing_skills	list[string]	

4.2 Synthetic Data Generator Script Logic

Generates `data/synthetic_students.csv`, 500 rows, using

realistic ranges and a **weighted formula to derive a ground-truth placement label** (needed so the classifier has something meaningful to learn — pure random labels will make SHAP output meaningless).

```
# backend/ml/data_gen.py
import numpy as np
import pandas as pd

np.random.seed(42)
N = 500

DEPARTMENTS = ["CSE", "ISE", "ECE", "MECH", "CIVIL"]
LANGUAGES = ["Python", "Java", "C++", "JavaScript", "SQL"]
CERTIFICATIONS = ["AWS", "Azure", "DBMS", "DSA", "Full-St
TARGET_ROLES = ["Full-Stack Developer", "Data Analyst", "

def generate_student(i):
    cgpa = np.clip(np.random.normal(7.2, 1.1), 4.0, 10.0)
    tenth = np.clip(np.random.normal(82, 8), 45, 100)
    twelfth = np.clip(np.random.normal(78, 9), 40, 100)
    backlogs = max(0, int(np.random.poisson(0.8)))
    projects = max(0, int(np.random.poisson(2.5)))
    internships = max(0, int(np.random.poisson(0.7)))
    aptitude = np.clip(np.random.normal(65, 15), 20, 100)
    soft_skill = np.clip(np.random.normal(6.5, 1.5), 1, 10)
    hackathons = max(0, int(np.random.poisson(1.0)))
    leadership = max(0, int(np.random.poisson(0.4)))
    open_source = max(0, int(np.random.poisson(0.5)))

    n_certs = np.random.randint(0, 4)
    certs = list(np.random.choice(CERTIFICATIONS, size=n_certs))
    n_langs = np.random.randint(1, 4)
    langs = list(np.random.choice(LANGUAGES, size=n_langs))

    # Weighted ground-truth score (0-100) used to derive
    score = (
        cgpa * 6.5 +
        (projects * 4) +
        (internships * 8) +
```

```

        (len(certs) * 5) +
        (aptitude * 0.25) +
        (soft_skill * 3) +
        (hackathons * 2) +
        (leadership * 2) +
        (open_source * 3) -
        (backlogs * 7)
    )
    score = np.clip(score + np.random.normal(0, 6), 0, 10)

    target_role = np.random.choice(TARGET_ROLES)

    return {
        "student_id": f"STU_{i:04d}",
        "department": np.random.choice(DEPARTMENTS),
        "semester": np.random.randint(3, 9),
        "cgpa": round(cgpa, 2),
        "tenth_pct": round(tenth, 1),
        "twelfth_pct": round(twelfth, 1),
        "backlogs": backlogs,
        "known_languages": "|".join(langs),
        "certifications": "|".join(certs),
        "projects_count": projects,
        "internships_count": internships,
        "open_source_contributions": open_source,
        "aptitude_score": round(aptitude, 1),
        "soft_skill_rating": round(soft_skill, 1),
        "hackathons_attended": hackathons,
        "leadership_roles": leadership,
        "target_role": target_role,
        "placement_probability_label": round(score, 1),
    }

def generate_dataset(path="backend/data/synthetic_student
    rows = [generate_student(i + 1) for i in range(N)]
    df = pd.DataFrame(rows)
    df.to_csv(path, index=False)
    print(f"Generated {len(df)} rows -> {path}")
    return df

```

```
if __name__ == "__main__":  
    generate_dataset()
```

4.3 ML Model Pipeline Specification

4.3.1 Feature Engineering & Preprocessing

```
# backend/ml/train_model.py (feature engineering excerpt)  
import pandas as pd  
  
FEATURE_COLUMNS = [  
    "cgpa", "tenth_pct", "twelfth_pct", "backlogs",  
    "projects_count", "internships_count", "open_source_c",  
    "aptitude_score", "soft_skill_rating", "hackathons_at",  
    "leadership_roles", "num_certifications", "num_language"  
]  
  
def engineer_features(df: pd.DataFrame) -> pd.DataFrame:  
    df = df.copy()  
    df["num_certifications"] = df["certifications"].apply(  
        lambda x: 0 if pd.isna(x) or x == "" else len(x.split(", "))  
    )  
    df["num_languages"] = df["known_languages"].apply(  
        lambda x: 0 if pd.isna(x) or x == "" else len(x.split(", "))  
    )  
    return df[FEATURE_COLUMNS]
```

- All numeric — no categorical encoding needed for the core feature set (department/target_role handled as separate classification heads if time allows; MVP treats them as filters, not model inputs, to keep SHAP interpretation clean).
- No scaling required for tree-based models (Random Forest / XGBoost are scale-invariant).
- Train/test split: 80/20, `random_state=42` for reproducibility during the demo.

4.3.2 Classifier Training

```
# backend/ml/train_model.py (continued)
import joblib
import json
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
# Swap to xgboost.XGBRegressor if the library installs cl

from data_gen import generate_dataset

def train():
    df = generate_dataset()
    X = engineer_features(df)
    y = df["placement_probability_label"] # continuous 0

    X_train, X_test, y_train, y_test = train_test_split(X

    model = RandomForestRegressor(
        n_estimators=200, max_depth=8, min_samples_leaf=4
    )
    model.fit(X_train, y_train)

    score = model.score(X_test, y_test)
    print(f"Validation R^2: {score:.3f}")

    joblib.dump(model, "backend/ml/artifacts/placement_mo
    with open("backend/ml/artifacts/feature_columns.json"
        json.dump(FEATURE_COLUMNS, f)

    return model

if __name__ == "__main__":
    train()
```

Design note: Placement probability (0–100%) is modeled as a **regression** target rather than a binary classifier, since the problem statement explicitly wants a continuous score, not just Ready/Not-

Ready. Readiness status (Ready / Near-Ready / Needs Training) is then derived from the continuous score via fixed thresholds (≥ 75 , $60-74$, < 60) — this also makes the SHAP explanation additive and directly interpretable in percentage-point terms.

4.3.3 Target Role Mapping

Simple confidence scoring per role (no separate model needed for MVP): compute a **benchmark-match percentage** per role by comparing the student's `known_languages / certifications / cgpa / projects_count` against each role's `required_skills` and minimums in `role_benchmarks.json`. Rank roles by match %.

4.4 Explainable AI (XAI) Implementation

```
# backend/ml/explainer.py
import shap
import joblib
import json
import pandas as pd

model = joblib.load("backend/ml/artifacts/placement_model")
with open("backend/ml/artifacts/feature_columns.json") as f:
    FEATURE_COLUMNS = json.load(f)

explainer = shap.TreeExplainer(model)

FEATURE_LABELS = {
    "cgpa": "CGPA",
    "backlogs": "backlogs",
    "projects_count": "project experience",
    "internships_count": "internships",
    "open_source_contributions": "open-source contributions",
    "aptitude_score": "aptitude performance",
    "soft_skill_rating": "soft skills / communication",
```

```

    "hackathons_attended": "hackathon participation",
    "leadership_roles": "leadership experience",
    "num_certifications": "certifications",
    "num_languages": "language/framework breadth",
    "tenth_pct": "10th percentage",
    "twelfth_pct": "12th percentage",
}

def explain_student(feature_row: pd.DataFrame, top_n: int)
    """
    feature_row: single-row DataFrame with columns == FEA
    Returns a list of dicts sorted by absolute impact, e.
    {"feature": "internships_count", "impact_pct": 18.4,
     "readable_string": "+18.4% due to internship experie
    """
    shap_values = explainer.shap_values(feature_row)[0]

    impacts = []
    for feat, val in zip(FEATURE_COLUMNS, shap_values):
        direction = "positive" if val >= 0 else "negative"
        sign = "+" if val >= 0 else "-"
        label = FEATURE_LABELS.get(feat, feat)
        readable = f"{sign}{abs(val):.1f}% due to {label}"
        impacts.append({
            "feature": feat,
            "impact_pct": round(float(val), 2),
            "direction": direction,
            "readable_string": readable,
        })

    impacts.sort(key=lambda x: abs(x["impact_pct"]), reverse=True)
    return impacts[:top_n]

```

- `shap.TreeExplainer` is used specifically because it's **exact and fast** for tree ensembles (Random Forest / XGBoost) — no sampling approximation needed, which matters under a 16-hour clock where re-running LIME per-request would be far slower.
- Each SHAP value is already in the same units as the model output (percentage points of placement probability), so no extra

normalization step is required before formatting the readable string.

4.5 Skill Gap & Roadmap Logic

```
# backend/services/skill_gap_service.py
import json

with open("backend/data/role_benchmarks.json") as f:
    ROLE_BENCHMARKS = json.load(f)

PHASE_TIMELINE_WEEKS = {"foundational": 2, "intermediate": 2, "advanced": 2}

def compute_skill_gap(student_skills: list[str], target_role: str) -> dict[str, list[str]]:
    required = set(ROLE_BENCHMARKS[target_role]["required_skills"])
    have = set(student_skills)
    return sorted(required - have)

def generate_roadmap(missing_skills: list[str], target_role: str) -> dict[str, list[str]]:
    """
    Buckets missing skills into 3 phases based on the role's difficulty tier for each skill, and attaches a resource link.
    """
    tiers = ROLE_BENCHMARKS[target_role].get("skill_tiers")
    phases = {"foundational": [], "intermediate": [], "advanced": []}

    for skill in missing_skills:
        tier = tiers.get(skill, "foundational")
        phases[tier].append(skill)

    roadmap = []
    for phase_name, skills in phases.items():
        if not skills:
            continue
        roadmap.append({
            "phase": phase_name.capitalize(),
            "skills": skills,
            "resources": ROLE_BENCHMARKS[target_role].get("resources", {})
```

```

        "estimated_weeks": PHASE_TIMELINE_WEEKS[phase]
        "recommended_action": f"Complete a guided pro
    })
    return roadmap

```

role_benchmarks.json structure (minimum viable, hand-author ~4 roles):

```

{
  "Full-Stack Developer": {
    "required_skills": ["JavaScript", "SQL", "Full-Stack"]
    "min_cgpa": 6.5,
    "min_projects": 2,
    "min_apptitude_score": 55,
    "skill_tiers": {
      "SQL": "foundational",
      "DSA": "foundational",
      "JavaScript": "intermediate",
      "Full-Stack": "advanced"
    }
  }
}

```

5. PART C: COMPLETE API INTERFACE CONTRACT (OpenAPI-Style Spec)

Base URL: `http://localhost:8000`. All responses wrapped in the standard envelope shown in §1.5.

5.1 POST `/api/predict`

Request body:

```
{
  "student_id": "STU_0001",
  "department": "CSE",
  "semester": 6,
  "cgpa": 7.8,
  "tenth_pct": 88.0,
  "twelfth_pct": 82.0,
  "backlogs": 0,
  "known_languages": ["Python", "JavaScript", "SQL"],
  "certifications": ["AWS", "Full-Stack"],
  "projects_count": 3,
  "internships_count": 1,
  "open_source_contributions": 2,
  "aptitude_score": 72.0,
  "soft_skill_rating": 7.5,
  "hackathons_attended": 2,
  "leadership_roles": 0,
  "target_role": "Full-Stack Developer"
}
```

Response body (200):

```
{
  "success": true,
  "data": {
    "student_id": "STU_0001",
    "placement_probability": 78.4,
    "readiness_status": "Ready",
    "target_role_scores": {
      "Full-Stack Developer": 82.1,
      "Data Analyst": 61.3,
      "Cloud/DevOps Engineer": 55.0,
      "QA Specialist": 58.7
    },
    "shap_factors": [
      { "feature": "internships_count", "impact_pct": 18.
      { "feature": "num_certifications", "impact_pct": -1
```

```

],
"missing_skills": ["DSA"],
"roadmap": [
  {
    "phase": "Foundational",
    "skills": ["DSA"],
    "estimated_weeks": 2,
    "recommended_action": "Complete a guided project"
  }
]
},
"error": null
}

```

5.2 GET /api/tpo/analytics

Query params (optional): ?department=CSE&threshold=60

Response body (200):

```

{
  "success": true,
  "data": {
    "overall_readiness_pct": 64.2,
    "department_breakdown": [
      { "department": "CSE", "readiness_pct": 71.0, "stud
      { "department": "ISE", "readiness_pct": 66.5, "stud
      { "department": "ECE", "readiness_pct": 55.3, "stud
    ],
    "vulnerable_students": [
      {
        "student_id": "STU_0042",
        "department": "ECE",
        "placement_probability": 43.2,
        "readiness_status": "Needs Training",
        "top_missing_skill": "SQL"
      }
    ]
  }
}

```

```

    ],
    "skill_deficit_matrix": {
      "departments": ["CSE", "ISE", "ECE", "MECH", "CIVIL"],
      "skills": ["SQL", "Python", "AWS", "DSA"],
      "matrix": [
        [12.0, 8.0, 40.0, 15.0],
        [20.0, 10.0, 55.0, 25.0],
        [64.0, 45.0, 70.0, 60.0],
        [70.0, 80.0, 90.0, 75.0],
        [66.0, 78.0, 88.0, 71.0]
      ]
    }
  },
  "error": null
}

```

(matrix[i][j] = % of students in departments[i] lacking skills[j].)

5.3 POST /api/roadmap

Use this as a **standalone** endpoint if the frontend wants to regenerate/refresh a roadmap independently of a full prediction (e.g., after the student manually edits their skill list).

Request body:

```

{
  "student_id": "STU_0001",
  "known_skills": ["Python", "JavaScript", "SQL", "AWS",
  "target_role": "Full-Stack Developer"
}

```

Response body (200):

```

{
  "success": true,

```

```
"data": {
  "student_id": "STU_0001",
  "missing_skills": ["DSA"],
  "roadmap": [
    {
      "phase": "Foundational",
      "skills": ["DSA"],
      "estimated_weeks": 2,
      "recommended_action": "Complete a guided project"
    }
  ]
},
"error": null
}
```

Error response shape (any endpoint, 4xx/5xx):

```
{
  "success": false,
  "data": null,
  "error": "cgpa must be between 0.0 and 10.0"
}
```

6. PART D: STEP-BY-STEP PROMPTING GUIDE FOR AI BUILDERS

Use these sequentially. Paste one block at a time and let the tool finish before pasting the next — bundling all steps into one giant prompt produces worse, harder-to-debug output under time pressure.

6.1 Developer 1 → Bolt.new / v0.dev (Frontend Generation)

Prompt 1 — Scaffold:

```
Create a React + Vite + Tailwind CSS project called "ai-p
Set up React Router v6 with two routes: "/student" and "/"
with links between them. Install and configure @tanstack/
Use a color palette: brand indigo (#4f46e5), status green
red (#dc2626). Rounded-2xl cards with subtle shadow and b
```

Prompt 2 — Student Diagnostics View:

```
On the "/student" route, build a "Student Diagnostics Vie
1. A toggle between "Manual Entry" (form) and "CSV Upload
   using PapaParse to preview parsed rows in a small tabl
2. The manual form fields: department (select), semester
   10th % , 12th % , backlogs (number), known languages (m
   certifications (multi-select tags), projects count, in
   open source contributions, aptitude score (0-100), sof
   hackathons attended, leadership roles, target role (se
   Full-Stack Developer, Data Analyst, Cloud/DevOps Engin
3. A "PlacementProbabilityGauge" component: a radial/circ
   colored red under 40, amber 40-60, green above 60, wit
   ("Ready" / "Near-Ready" / "Needs Training") underneath
4. A "FactorContributionChart": horizontal bar chart wher
   one feature's impact (positive bars green, negative ba
   bar's text label showing a readable impact string like
   "+18% due to internships".
5. An "UpskillingRoadmap" component: a vertical phased ti
   Phase 1 / Phase 2 / Phase 3, each with a list of skill
   sentence, and an estimated weeks badge.
Use react-query's useMutation to POST the form data to PO
base URL import.meta.env.VITE_API_BASE_URL, and populate
components from the response. Show a loading spinner whil
dismissible red error banner on failure. Do not use brows
```

Prompt 3 — TPO Analytics Dashboard:

On the `"/tpo"` route, build a "TPO Analytics Dashboard" page.

1. A responsive grid of "Department Readiness" cards (one for each department: CSE, ISE, ECE, MECH, CIVIL), each showing the department name, average readiness percentage, student count, and a horizontal progress bar. Use the same red/amber/green thresholds as the gauge.
2. A "Cohort Vulnerability Table": sortable table with columns: Department, Placement Probability, Readiness Status (Complete, In Progress, Missing Skill). Default view filters to students below 70% readiness. Add a toggle switch to show all students.
3. A "Skill Deficit Heatmap": a grid where rows are departments and columns are skills, and each cell's background color intensity represents the percentage of students in that department lacking that skill (higher deficit = darker red). Show the percentage as text inside each cell.

Fetch all data from `GET /api/tpo/analytics` using `react-query`. Implement pagination (10 items per page), with loading skeletons and an error state for each section. Update sections independently (don't let one section's failure block the others).

6.2 Developer 2 → ChatGPT / Claude (Backend Generation)

Prompt 1 — `data_gen.py`:

Write a Python script `data_gen.py` using `numpy` and `panda` to generate a synthetic dataset of 500 students for a placement-readiness dashboard. The dataset should have the following columns: `student_id`, `department` (CSE/ISE/ECE/MECH/CIVIL), `cgpa` (4.0-10.0, normal dist mean 7.2), `tenth_pct`, `twelfth_pct` (poisson, mostly 0-2), `known_languages` (pipe-delimited, 1-5, e.g., C++/JavaScript/SQL/Go), `certifications` (pipe-delimited, 0-5, e.g., AWS/DBMS/DSA/Full-Stack/Data Science/Cybersecurity), `projects_completed` (mean 2.5), `internships_count` (poisson mean 0.7), `open_source_contributions`, `aptitude_score` (0-100), `soft_skill_rating` (1-10), `hackathons`, `leadership_roles`, `target_role` (one of Full-Stack Developer, Frontend Engineer, Backend Engineer, Cloud-DevOps Engineer/QA Specialist), and a derived column `placement_probability_label` (0-100) computed as a weighted average of `cgpa`, `aptitude_score`, `soft_skill_rating`, and `projects_completed`.

positive features minus a penalty for backlogs, plus gaussian noise to $[0,100]$. Save to `backend/data/synthetic_students.csv`.

Prompt 2 — `model.py` / `train_model.py`:

Write a Python training script ``train_model.py`` that loads `backend/data/synthetic_students.csv`, engineers numeric features (e.g. `num_certifications` and `num_languages` as counts from pipe-delimited columns), splits 80/20, and trains a `sklearn.ensemble.RandomForestRegressor` (200 estimators, `min_samples_leaf` 4, `random_state` 42) to predict placement as a continuous 0-100 score. Print the validation R^2 . Save the trained model with `joblib` to `backend/ml/artifacts/placement_model.pkl`. Save an ordered list of feature column names as JSON to `backend/ml/artifacts/feature_columns.json`. Also write a `predict_student(feature_dict) -> float` function that loads the saved model and feature column list, builds a single-row DataFrame with the correct column order, and returns the predicted probability.

Prompt 3 — `explainer.py` (SHAP):

Write ``explainer.py`` that loads the `joblib` model and feature columns from `backend/ml/artifacts/`, builds a `shap.TreeExplainer` and exposes ``explain_student(feature_row_df, top_n=5)`` which takes a single-row input DataFrame, maps each raw feature name to a human-readable label (e.g. `"internships_count"` -> `"internships"`, `"num_certifications"` -> `"certifications"`), and returns the top `top_n` features sorted by absolute impact as a list of dicts with keys: `feature` (the raw SHAP value rounded to 1 decimal), `direction` (`"positive"` or `"negative"`) and `readable_string` formatted like `"+18.4% due to internships"` or `"-14.0% due to missing certifications"` (use `"missing"` phrase if `direction` is negative).

Prompt 4 — `main.py` (FastAPI app + all three routes):

Write a FastAPI application ``main.py`` with CORS enabled for `http://localhost:5173`, that registers three routers:

1. `POST /api/predict` - accepts a Pydantic model matching

```
student_id: str, department: str, semester: int, cgpa:
tenth_pct: float, twelfth_pct: float, backlogs: int, k
list[str], certifications: list[str], projects_count:
internships_count: int, open_source_contributions: int
float, soft_skill_rating: float, hackathons_attended:
leadership_roles: int, target_role: str.
```

It should engineer features, call `predict_student()` to derive `readiness_status` from thresholds (≥ 75 Ready, $60 < 75$ Needs Training), call `explain_student()` for `shap_f` missing skills by comparing known skills against `role_target_role_scores` as a benchmark-match percentage against `role_benchmarks.json`. Return everything wrapped in `{"success": true, "data": {...}, "error": null}`.

2. GET `/api/tpo/analytics` - accepts optional query params `threshold` (default 60). Loads `synthetic_students.csv`, across all rows to get each student's probability and `overall_readiness_pct`, a `department_breakdown` list, a list filtered below the threshold, and a `skill_deficit` departments, columns = a fixed skill list, values = % department missing that skill relative to a generic full list). Wrap in the same success/data/error envelope.
3. POST `/api/roadmap` - accepts `student_id`, `known_skills`: `target_role: str`, and returns `missing_skills` and a phase the same `skill_gap_service` logic as endpoint 1, in the Raise `HTTPException(422)` with a clear message wrapped as `{"success": false, "data": null, "error": "<message>"}` on validation failure (e.g. cgpa out of range). Include a route check that returns `{"status": "ok"}`.

Build Order Recommendation (for the 16-hour clock)

1. **Hour 0–1:** Both devs align on this doc + the exact JSON shapes in Part C. Do not deviate from the field names without updating both sides.

2. **Hour 1–5 (Dev 2):** `data_gen.py` → `train_model.py` → `model.py` → `explainer.py`, verified in a notebook/script before touching FastAPI.
3. **Hour 1–5 (Dev 1):** Scaffold + both views built against hard-coded mock JSON matching Part C exactly.
4. **Hour 5–9:** Dev 2 wraps ML in FastAPI routes; Dev 1 wires `ApiClient` to the real base URL and swaps mocks for live hooks.
5. **Hour 9–12:** Skill-gap/roadmap logic + TPO aggregation; frontend heatmap/table polish.
6. **Hour 12–14:** Integration pass — run both together, fix contract mismatches.
7. **Hour 14–16:** Styling polish, edge-case handling (empty states, bad CSV), demo script rehearsal.